Dauphine | PSL
UNIVERSITÉ PARIS

MIDO
MATHÉMATIQUES ET INFORMATIQUE
DE LA DÉCISION ET DES ORGANISATIONS

M2 IASD

Large Language Models
Topic – LlamaFactory: Unified Efficient Fine-Tuning of 100+ Language Models

Sepanta Farzollahi, Mellissa Hafis

Instructor: Shuyu Dong

June 18, 2026

Abstract

Fine-tuning large language models (LLMs) for downstream tasks is computationally prohibitive at full scale, motivating a growing family of parameter-efficient methods. This report analyzes and partially replicates *LlamaFactory* [16], a unified framework that integrates more than ten efficient fine-tuning methods across 100+ LLMs under a single interface. We reproduce the qualitative patterns of Table 4 of the original paper on TinyLlama-1.1B-Chat [14] with the Alpaca GPT-4 dataset [12] on a single NVIDIA RTX 6000 Ada GPU, comparing Full fine-tuning, Freeze-tuning, GaLore, LoRA, and QLoRA together with a carefully calibrated zero-shot baseline. We additionally contribute a novel data-size ablation studying LoRA vs. QLoRA across six training set sizes from 100 to 10,000 examples, and document two important implementation details absent from the original paper.

1 Introduction

Adapting pre-trained LLMs to specific downstream tasks requires fine-tuning, but updating all parameters of a 7B-parameter model demands approximately 38 GB of GPU memory in half precision, which is prohibitive for most practitioners. This has motivated a rich line of parameter-efficient fine-tuning (PEFT) research: LoRA [5], QLoRA [2], GaLore [15], and others. However, fair comparison across these methods has been difficult since each is typically implemented with a different codebase, dataset format, and evaluation protocol.

LlamaFactory [16] addresses this by providing a single, model-agnostic framework supporting 100+ LLMs and all major efficient fine-tuning techniques through a consistent YAML-based interface. The paper received over 25,000 GitHub stars and has been used to build hundreds of open-source models on Hugging Face Hub, attesting to its practical impact. In this project we (1) analyze the architecture and technical mechanisms of the LlamaFactory framework in depth, going beyond what the paper itself details, (2) replicate the efficiency comparison of Table 4 on TinyLlama-1.1B, and (3) contribute a novel data-size ablation together with two undocumented implementation findings.

2 The LlamaFactory Framework

LlamaFactory is built around three loosely coupled modules: a **Model Loader**, a **Data Worker**, and a **Trainer**, together with an optional web UI (**LlamaBoard**) built on Gradio. The separation of concerns between these modules is what enables the framework to scale to 100+ models and dozens of fine-tuning methods without requiring model-specific code. It is worth noting that the paper, published as a system demonstration at ACL 2024, describes the framework at a high level and contains no mathematical formulations. The internal mechanisms of each module are not detailed in the paper itself; the descriptions below are reconstructed from the framework's source code, the referenced libraries (HuggingFace Transformers, PEFT, bitsandbytes), and the original method papers.

2.1 Overall Design and Supported Methods

The paper focuses its Table 4 comparison on five representative optimization methods, but the framework supports a broader set organized into two categories.

Optimization methods (which parameters to update and how gradients flow): Full fine-tuning, Freeze-tuning [4], GaLore [15], BAdam [7], LoRA [5], LoRA+ [3], DoRA [6], PiSSA [9], rsLoRA, and QLoRA [2].

Computation methods (how to reduce the cost of each update): mixed precision training (fp16/bf16), activation checkpointing, Flash Attention [1], S^2-attention for long contexts, 4-bit and 8-bit quantization (bitsandbytes, GPTQ, AWQ, AQLM), and Unsloth, a Triton-based implementation of the LoRA backward pass that reduces floating-point operations during gradient descent.

These computation methods can be combined with any optimization method, except quantization which is restricted to adapter-based methods, since the quantized base model weights cannot be directly fine-tuned.

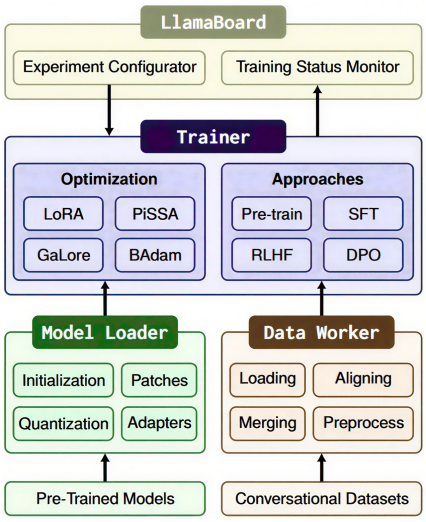

Figure 1: Architecture of LlamaFactory.

2.2 Model Loader

The Model Loader is responsible for preparing the pre-trained model for training. The paper mentions four components (initialization, patching, quantization, and adapter attaching) but does not explain the mechanisms behind them.

Initialization. Models are loaded via HuggingFace `AutoModelForCausalLM` [13] (or `AutoModelForVision2Seq` for vision-language models). When the tokenizer vocabulary exceeds the model embedding dimension, which can happen when adding domain-specific tokens, the embedding matrix is resized and new rows are initialized with *noisy mean initialization*: the mean of existing embeddings plus Gaussian noise, which avoids disrupting the geometric structure of the existing token representations.

Precision adaptation. The paper simply states that precision is handled "based on the capabilities of computing devices" without further detail. In practice, LlamaFactory uses bfloat16 on GPUs with CUDA compute capability ≥ 8.0 (A100, RTX 4000 series and above), because bfloat16 has the same memory footprint as float16 but a larger dynamic range, reducing the risk of overflow during training. On older GPUs and on AMD/Ascend hardware, float16 is used. On CPU, float32 is used. Crucially, in mixed-precision training, *trainable parameters are always cast to float32* regardless of the base precision, to avoid gradient underflow that would otherwise corrupt updates of small-magnitude adapter weights.

Model patching. Flash Attention [1] is enabled via HuggingFace Transformers' native support. The standard attention kernel, which requires memory proportional to the square of the sequence length $O(n^2)$, is replaced with a block-based implementation that reduces this to $O(n)$, making long-sequence training feasible. For S^2-attention (shifted sparse attention for long-context fine-tuning), LlamaFactory applies a code-level patch at the forward function since this variant lacks native Transformers support.

Adapter attachment and quantization. The paper states that adapters are "automatically identified" and attached to appropriate layers, but gives no further detail. In practice, adapters are attached to *all* linear layers in the model rather than only the attention projections, following the finding of Dettmers et al. [2] that full-layer coverage leads to better convergence. The PEFT library [8] handles the low-level implementation. For QLoRA, base model weights are stored in 4-bit NF4 (Normal Float 4) format. NF4 is designed specifically for normally distributed weights: unlike standard INT4, which uses evenly spaced quantization bins, NF4 places more bins near zero where neural network weights are densest, minimizing quantization error under a normality assumption. Double quantization is applied on top: the 32-bit quantization constants (one per 64-weight block) are themselves quantized to 8-bit, saving approximately 0.37 additional bits per parameter. During the forward pass, base weights are dequantized on-the-fly to float16 before each matrix multiplication; adapter matrices A and B remain in float16 throughout.

2.3 Data Worker

The paper describes the Data Worker as handling dataset loading, aligning, merging, and pre-processing, with the mention that "chat template is a crucial component" but does not explain how the template selection works or what its consequences are for evaluation. The Data Worker standardizes datasets of heterogeneous formats (Alpaca, ShareGPT, plain text, preference data) into a unified internal representation.

The most consequential design choice is the **chat template**. LlamaFactory maintains dozens of templates and auto-selects the correct one based on the model name. For TinyLlama-1.1B-Chat, it selects the Zephyr template:

```
<|system|>{system}</s>\n<|user|>{instruction}</s>\n<|assistant|>{response}</s>
```

This matters for two reasons. First, a model trained with one template will perform poorly when evaluated with another, so the framework must be consistent between training and evaluation. Second, as we discuss in Section 4.2, using the wrong template when computing a pre-trained model's perplexity produces values that are orders of magnitude too high.

By default, the cross-entropy loss is computed *only on response tokens*. The paper mentions this briefly ("we only compute loss on the completions") but does not describe the mechanism. Concretely, the data collator builds a `labels` tensor that is identical to `input_ids` for the assistant turn and -100 everywhere else, a value that PyTorch's `CrossEntropyLoss` silently ignores. This design reflects the Supervised Fine-Tuning (SFT) objective: the model should learn to generate answers given instructions, not to reconstruct instructions it already has access to.

2.4 Trainer

The paper describes the Trainer as integrating efficient fine-tuning methods "by replacing default components", and notes that methods are "independent of the Trainer", but does not detail how this plug-and-play mechanism works internally. The Trainer supports four objectives: generative pre-training, SFT (both using HuggingFace Trainer), RLHF (via TRL's PPO implementation [10]), and DPO (via TRL's DPO trainer [11]).

Plug-and-play optimizer injection. Methods like GaLore, LoRA+, and BAdam are integrated by overriding only the `create_optimizer` method of HuggingFace Trainer, leaving the training loop, checkpointing, and evaluation logic unchanged. This is the key architectural decision that enables new methods to be supported without modifying the entire training pipeline. For GaLore specifically, the custom optimizer creates per-layer `GaLoreAdamW` instances that wrap standard AdamW with gradient projection hooks applied before each parameter update. We discovered that this mechanism is incompatible with the fused AdamW kernel (`adamw_torch_fused`) that LlamaFactory selects by default on modern GPUs, an incompatibility not documented in the official README (see Appendix B).

Model-sharing RLHF. One notable contribution of the Trainer is enabling RLHF on consumer hardware. Standard RLHF requires four distinct models (policy, reference, reward, and value models), demanding 4× the base model size in GPU memory. LlamaFactory reduces this to a single pre-trained model by dynamically switching between multiple adapters and value heads using PEFT's `set_adapter` and `disable_adapter` methods, making full RLHF accessible on a single GPU.

3 Efficient Fine-Tuning Methods

Since LlamaFactory provides no mathematical formulations for the methods it implements, we reconstruct the core mechanisms here from the original papers. For each method, we describe both the update rule and the associated memory cost, since the memory–quality trade-off is the central comparison in the original paper.

Full fine-tuning minimizes the standard language modelling cross-entropy loss:

$$\mathcal{L}(\theta) = -\sum_t \log p_\theta(x_t \mid x_{<t})$$

where θ denotes all model parameters, x_t is the token at position t, and $p_\theta(x_t \mid x_{<t})$ is the model's predicted probability for that token given the preceding context. All N parameters are updated at each step. Memory cost is approximately $10N$ bytes: $2N$ for fp16 weights plus $8N$ for Adam's two float32 momentum terms (first and second moments).

Freeze-tuning [4] restricts updates to the last k transformer blocks, while all earlier layers remain frozen. Only the trainable weights accumulate optimizer states, reducing memory proportionally to the fraction of active parameters (176M out of 1,100M, i.e. ≈16%, in our setup). The trade-off is that earlier layers encoding general linguistic structure are not adapted, potentially limiting expressivity on tasks requiring deep structural changes.

GaLore [15] trains all parameters but projects gradients onto a low-rank subspace before the optimizer step. For a weight matrix $W \in \mathbb{R}^{m\times n}$ (where m and n are the output and input dimensions of a given linear layer), the gradient $G = \nabla_W \mathcal{L}$ is approximated as:

$$\tilde{G} = PP^\top G, \quad P \in \mathbb{R}^{m\times r},\ r \ll m$$

where P holds the top-r left singular vectors of G obtained via Singular Value Decomposition (SVD), and r is the projection rank (set to 16 in our experiments). Adam's first and second momentum terms are maintained only in this r-dimensional subspace, reducing optimizer-state memory from $O(mn)$ to $O(mr+nr)$ per layer. The projection matrix P is recomputed every T steps (every 200 steps here). Since all parameters can be updated (only the gradient direction is constrained to a low-rank subspace), GaLore retains more expressivity than LoRA in principle.

LoRA [5] freezes the pre-trained weight matrix $W_0 \in \mathbb{R}^{d\times k}$ and introduces a pair of trainable low-rank matrices. The modified forward pass for a layer with input x and output h is:

$$h = W_0 x + \frac{\alpha}{r} BAx$$

where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$ are the adapter matrices, $r \ll \min(d,k)$ is the adapter rank (64 in our experiments), and α is a scaling hyperparameter (128 in our experiments). B is initialized to zero and A with a random Gaussian, so the perturbation $\Delta W = BA = 0$ at initialization, meaning training starts from the pre-trained model behavior. The number of trainable parameters per layer is $r(d+k)$ instead of dk; at rank 64 applied to all linear layers of TinyLlama-1.1B, only 18M parameters are trainable, which is less than 2% of the total 1.1B.

QLoRA [2] combines LoRA with 4-bit NF4 quantization of the frozen base weight matrix W_0. The NF4 format places quantization bins at the quantiles of the standard normal distribution rather than at evenly spaced intervals, minimizing expected quantization error under the assumption that neural network weights follow an approximately normal distribution. Double quantization is applied on top: the per-block float32 scaling constants (one per group of 64 weights) are themselves quantized to 8-bit, saving ∼0.37 additional bits per parameter. During the forward pass, W_0 is dequantized on-the-fly from NF4 to float16 for each matrix multiplication; the adapter matrices A and B remain in float16 throughout. The total memory saving for the base model relative to float16 LoRA is approximately 4×.

4 Experiments

We replicate the efficiency comparison of Table 4 from Zheng et al. [16] on a smaller model and an instruction-following task. All experiments are run using LlamaFactory's YAML interface with no modifications to the framework source code.

4.1 Setup

Table 1 compares our setup with the original paper.

	Original paper	**Our replication**
Model	Gemma-2B, Llama2-7/13B	TinyLlama-1.1B-Chat
Dataset	PubMed (∼400K tokens)	Alpaca GPT-4 (10K examples)
Objective	Generative pre-training	Supervised fine-tuning
GPU	NVIDIA A100 40 GB	NVIDIA RTX 6000 Ada
Precision	bfloat16	float16
Optimizer	8-bit AdamW	AdamW (standard)

Table 1: Experimental setup comparison.

All methods share: 3 epochs, 512-token max length, cosine LR schedule with 10% warmup, evaluation every 100 steps on a 10% held-out validation split, and best-checkpoint selection by validation loss. LoRA/QLoRA: rank 64, $\alpha = 128$, dropout 0.05, all linear layers. Freeze: last 4 transformer blocks. GaLore: rank 16, scale 0.25, projection interval 200 steps, `optim:adamw_torch` (see Appendix B for why this is necessary).

4.2 Baseline Perplexity

A naive evaluation of the pre-trained model on the validation set gives Perplexity (PPL) ≈90,000. Three fixes are required to match LlamaFactory's internal evaluation protocol:

1. **Padding masking.** Batch-padding introduces tokens that the model cannot predict. Setting `labels[attention_mask==0]=-100` excludes them from the cross-entropy loss.
2. **Correct chat template.** The Alpaca format uses special markers of the form `### Instruction:` and `### Response:` to delimit the instruction and answer parts of the prompt. TinyLlama-1.1B-Chat was never trained with this format; it expects the Zephyr template instead. Evaluating with the wrong format confronts the model with token sequences it has never seen, artificially inflating its perplexity.
3. **Response-only loss.** LlamaFactory's SFT data collator masks instruction tokens. We replicate this by tokenizing the prompt-only prefix separately to obtain its length L_{prompt} and setting `labels[j, :`L_{prompt}`]=-100` per sample.

After all three fixes, the baseline perplexity is **PPL = 3.51**, a realistic zero-shot instruction-following score for a 1.1B chat model.

4.3 Main Results

Method	Trainable	Memory (GB)	Throughput (tok/s)	Eval Loss	PPL
Baseline	/	/	/	/	3.51
Full-FT	1,100M	24.35	6,981	1.109	3.03
Freeze	176M	9.81	16,836	1.134	3.11
GaLore	1,100M	22.04	6,838	1.123	3.07
LoRA	18M	8.02	10,440	1.117	3.06
QLoRA	18M	8.02	10,454	1.117	3.06

Table 2: Main experiment. TinyLlama-1.1B, 10,000 Alpaca GPT-4 examples, 3 epochs. Best validation checkpoint perplexity reported.

All three qualitative patterns from the original paper are confirmed. **QLoRA uses the least memory** (8.02 GB, 3× less than Full-FT and 2.7× less than GaLore) while matching LoRA's quality exactly. **LoRA achieves the best quality–efficiency trade-off**: PPL 3.06 with only 18M trainable parameters, within 1% of Full-FT (3.03) while using 3× less memory. **Freeze achieves the highest throughput** (16,836 tok/s, 2.4× faster than Full-FT) by eliminating gradient computation for frozen layers, but at the cost of the highest final perplexity (3.11) among fine-tuned models.

One result differs from the original paper: at 1.1B scale, **LoRA and QLoRA are exactly equal in perplexity** (both 3.06). The paper reports a gap of 0.27 PPL between LoRA and QLoRA on Gemma-2B (10.19 vs. 10.46). This is consistent with the known behavior of NF4 quantization: the relative quality impact of 4-bit noise decreases as the total number of quantized weights decreases. At 1.1B parameters, quantization introduces too little noise relative to the learning signal to produce a measurable quality gap.

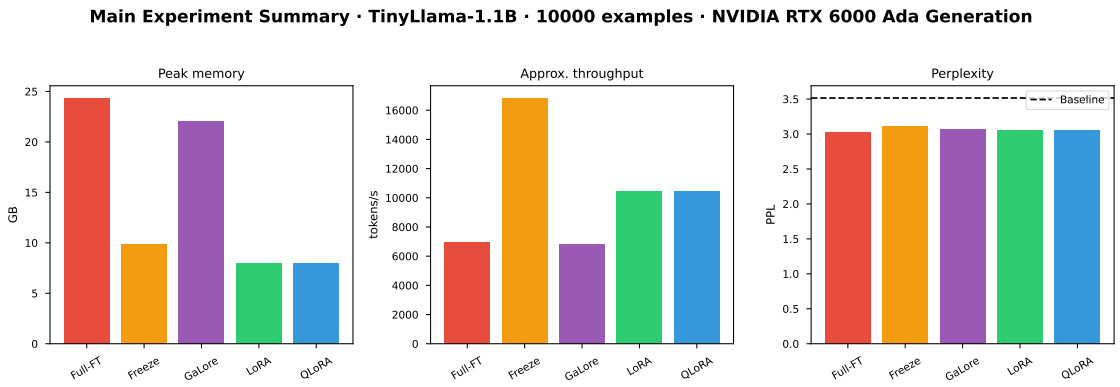

Figure 2: Memory, throughput, and perplexity for all five methods. LoRA and QLoRA dominate simultaneously on memory and quality.

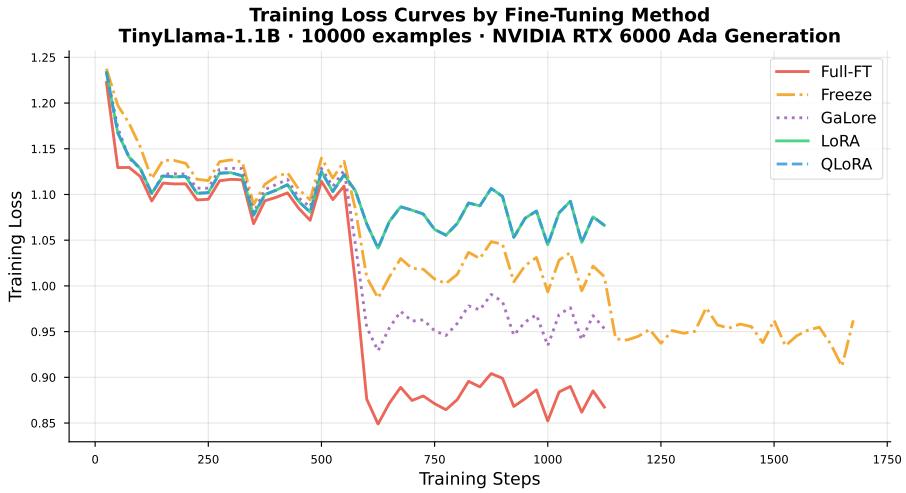

Figure 3: Training loss curves over 3 epochs. The sharp descent for Full-FT and GaLore at step ∼550 corresponds to the cosine scheduler reaching its minimum at the end of epoch 1. Freeze-tuning converges more slowly because only 16% of parameters are active, and its loss floor is higher.

4.4 Novel Ablation: LoRA vs. QLoRA Across Data Sizes

We investigate whether QLoRA's quantization noise becomes more harmful when training data is scarce, the intuition being that with fewer gradient updates, the model has less opportunity to compensate for precision loss. We train both methods at six scales: 100, 500, 1,000, 2,000, 5,000, and 10,000 examples.

Examples	LoRA PPL	QLoRA PPL	Gap (Q−L, eval loss)
100	3.18	3.18	−0.0003
500	3.28	3.28	≈ 0
1,000	3.05	3.05	≈ 0
2,000	2.93	2.93	≈ 0
5,000	3.03	3.03	≈ 0
10,000	3.06	3.06	≈ 0

Table 3: LoRA vs. QLoRA across training data sizes. Best validation checkpoint.

LoRA and QLoRA are **statistically indistinguishable at all six scales**, with a maximum absolute loss gap of 0.0003. Interestingly, at 100 examples QLoRA is marginally better (gap = −0.0003), which may indicate that 4-bit quantization acts as a mild implicit regularizer in extremely data-scarce regimes. The quantization-noise hypothesis from the original paper therefore holds only at larger model scales (≥2B parameters), where more weights are quantized and the signal-to-noise ratio of the NF4 approximation is lower.

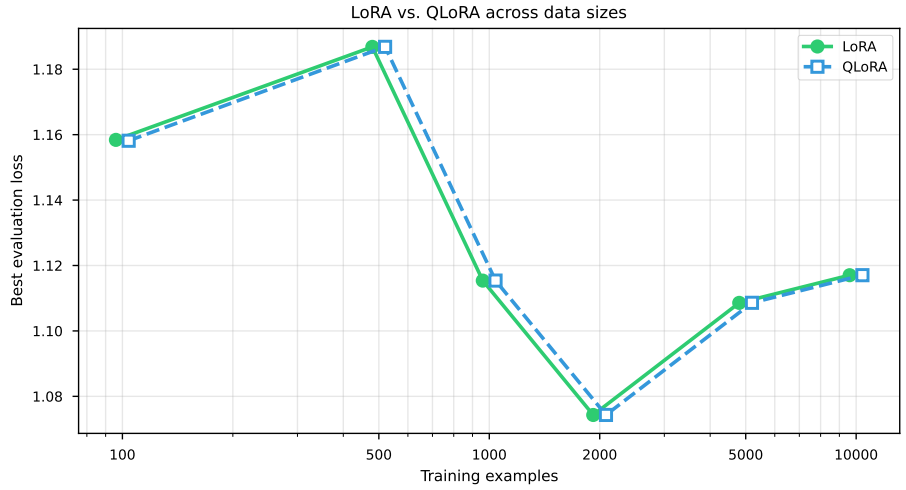

Figure 4: LoRA vs. QLoRA best validation loss across six data sizes. The two curves overlap at every scale.

5 Contributions

Replication of Table 4. We reproduce all qualitative patterns from the original paper across all five conditions using only LlamaFactory's YAML interface, without modifying framework source code. The full experimental notebook is provided with the submission.

Correct baseline evaluation. We identified and corrected three systematic errors that cause naive baseline PPL to reach ∼90,000: padding token leakage, wrong chat template, and loss computed over instruction tokens. The correct value is PPL = 3.51 (detailed in Appendix A).

GaLore compatibility fix. We discovered an undocumented incompatibility between LlamaFactory and GaLore on modern NVIDIA GPUs: LlamaFactory defaults to `adamw_torch_fused`, which GaLore's custom optimizer hook cannot handle, raising `NotImplementedError`. The fix is to add `optim:adamw_torch` to the YAML config (detailed in Appendix B).

Novel data-size ablation. We ran a new experiment comparing LoRA and QLoRA across six data sizes, finding that the two methods are equivalent in quality at 1.1B scale regardless of training data volume, a result not covered by the paper.

6 Limitations

Limitations of the original paper. The paper contains no mathematical formulations, making it insufficient as a standalone technical reference for the methods it implements. The internal mechanisms of the three modules are described only at a very high level, without explaining how precision selection, chat template matching, or optimizer injection work in practice. All efficiency results are from a single run without confidence intervals. Perplexity is evaluated on the training corpus (PubMed) rather than a held-out test set, conflating memorization with generalization. Additionally, Unsloth is enabled only for LoRA and QLoRA, creating a throughput confound: part of LoRA's speed advantage over GaLore may be attributable to Unsloth's Triton-optimized backward pass rather than LoRA itself.

Limitations of our replication. We use TinyLlama-1.1B on SFT rather than the paper's 2–13B models on generative pre-training, so absolute numbers are not comparable. Each condition is run once (single seed), so small observed differences carry no statistical guarantee. Throughput is estimated from wall-clock time and an assumed average sequence length of 256 tokens rather than measured directly. Models at 7B+ scale were not tested due to hardware constraints, limiting our ability to verify whether the LoRA–QLoRA equivalence we observe persists at the model sizes where the paper reports a gap.

7 Conclusion

LlamaFactory is a genuine and practically impactful engineering contribution. Its modular design (Model Loader, Data Worker, Trainer) allows any new fine-tuning method to be integrated by overriding a single optimizer factory method, without modifying the training loop. The framework supports a much broader set of methods than the five compared in Table 4, including DoRA, LoRA+, PiSSA, BAdam, Flash Attention, S^2-attention, and Unsloth. However, the paper itself remains deliberately high level: the internal mechanisms of each module are not explained, and there are no mathematical formulations for any of the methods integrated.

Our replication on TinyLlama-1.1B confirms all qualitative patterns from Table 4: QLoRA uses the least memory, LoRA achieves the best quality–efficiency trade-off, and Full fine-tuning maximizes quality at the highest memory cost. Our novel ablation shows that at 1.1B scale, LoRA and QLoRA are interchangeable in quality at all training data volumes tested. The practical recommendation is clear: **at sub-2B scale, QLoRA can be used freely without any quality penalty**, the gap reported in the original paper being a phenomenon specific to larger models.

References

[1] Tri Dao et al. "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness". In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 16344–16359.

[2] Tim Dettmers et al. "QLoRA: Efficient Fine-Tuning of Quantized LLMs". In: *Advances in Neural Information Processing Systems* 36 (2023), pp. 10088–10115.

[3] Soufiane Hayou, Nikhil Ghosh, and Bin Yu. "LoRA+: Efficient Low Rank Adaptation of Large Models". In: *International Conference on Machine Learning*. 2024.

[4] Neil Houlsby et al. "Parameter-Efficient Transfer Learning for NLP". In: *International Conference on Machine Learning*. 2019, pp. 2790–2799.

[5] Edward J Hu et al. "LoRA: Low-Rank Adaptation of Large Language Models". In: *International Conference on Learning Representations*. 2022.

[6] Shih-Yang Liu et al. "DoRA: Weight-Decomposed Low-Rank Adaptation". In: *International Conference on Machine Learning*. 2024.

[7] Qijun Luo, Hengxu Yu, and Xiao Li. "BAdam: A Memory Efficient Full Parameter Training Method for Large Language Models". In: *arXiv preprint arXiv:2404.02827*. 2024.

[8] Sourab Mangrulkar et al. *PEFT: State-of-the-Art Parameter-Efficient Fine-Tuning Methods*. 2022. URL: `https://github.com/huggingface/peft`.

[9] Fanxu Meng, Zhaohui Wang, and Muhan Zhang. "PiSSA: Principal Singular Values and Singular Vectors Adaptation of Large Language Models". In: *arXiv preprint arXiv:2404.02948* (2024).

[10] Long Ouyang et al. "Training Language Models to Follow Instructions with Human Feedback". In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 27730–27744.

[11] Rafael Rafailov et al. "Direct Preference Optimization: Your Language Model is Secretly a Reward Model". In: *Advances in Neural Information Processing Systems* 37 (2023).

[12] Rohan Taori et al. "Stanford Alpaca: An Instruction-Following LLaMA Model". In: *GitHub repository* (2023). URL: `https://github.com/tatsu-lab/stanford_alpaca`.

[13] Thomas Wolf et al. "Transformers: State-of-the-Art Natural Language Processing". In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. 2020, pp. 38–45.

[14] Peiyuan Zhang et al. *TinyLlama: An Open-Source Small Language Model*. 2024. URL: `https://github.com/jzhang38/TinyLlama`.

[15] Jiawei Zhao et al. "GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection". In: *International Conference on Machine Learning*. 2024.

[16] Yaowei Zheng et al. "LlamaFactory: Unified Efficient Fine-Tuning of 100+ Language Models". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*. Association for Computational Linguistics, 2024, pp. 400–410.

Appendix

A. Correct Baseline Perplexity Computation

Three fixes are required to match LlamaFactory's SFT evaluation protocol. Without them, the naive perplexity estimate is ∼90,000; after all three it is 3.51.

1. **Mask padding:** set `labels[attention_mask == 0] = -100` to exclude padding tokens from the cross-entropy loss.
2. **Correct template:** call `tokenizer.apply_chat_template` with the Zephyr format, which LlamaFactory auto-selects for TinyLlama-1.1B-Chat. Using the Alpaca format instead confronts the pre-trained model with token sequences it has never seen, artificially inflating its loss.
3. **Response-only loss:** tokenize the prompt-only prefix (system + user, with `add_generation_prompt=True`) to obtain its token length L_{prompt}, then set `labels[j, :`$L^{(j)}_{\texttt{prompt}}$`] = -100` for each sample j. This mirrors the SFT data collator behavior.

Final PPL: $\exp\left(\dfrac{\sum_{\text{batches}} \texttt{loss} \times n_{\text{valid}}}{\sum_{\text{batches}} n_{\text{valid}}}\right)$ where n_{valid} counts non-masked tokens per batch.

B. GaLore + LlamaFactory Compatibility Fix

LlamaFactory selects `adamw_torch_fused` by default on NVIDIA GPUs with CUDA compute capability ≥ 7.0. GaLore overrides the Trainer's `create_optimizer` with a custom function that instantiates per-layer `GaLoreAdamW` optimizers. `GaLoreAdamW` wraps standard `torch.optim.AdamW` with gradient projection hooks but does not implement support for the fused kernel variant, raising:

```
NotImplementedError:Unknown_optim:OptimizerNames.ADAMW_TORCH_FUSED.
```

Fix: add `optim:adamw_torch` to the GaLore YAML configuration file. This forces the non-fused standard AdamW and is fully compatible with GaLore's projection hook. This incompatibility is not documented in the official LlamaFactory README or GitHub issues.

C. Use of AI Assistants

As required by the project guidelines, we disclose our use of AI tools. Claude (Anthropic, claude-sonnet-4) was used as an AI assistant throughout this project. Its contributions include: generating the initial structure of the experimental notebook and iteratively debugging it (in particular the baseline perplexity computation and the GaLore configuration error); producing a first draft of the Beamer presentation; and assisting with the drafting and editing of this report. All numerical results, experimental decisions, and critical analyses are our own. The final notebook, presentation, and report were reviewed, corrected, and approved by both authors.